Guia de apresentação e estudo

Verificação adversarial com 3 agentes para reduzir falsos positivos na detecção de vulnerabilidades: auditoria de Belozerov et al. (EMSE, 2026) e teste de um pipeline com modelos abertos. Projeto da disciplina TAES, Centro de Informática da UFPE, novembro de 2026.

Como usar este guia

O guia tem cinco partes. Leia a Parte 1 juntos, no primeiro ensaio. Cada pessoa estuda o seu bloco na Parte 2. Todos leem a Parte 3 e a Parte 4, porque a banca pode perguntar qualquer coisa a qualquer um.

1. **Plano da apresentação:** a mensagem central, a divisão do tempo, as passagens entre blocos e o checklist de ensaio.
2. **Roteiro slide a slide:** o que dizer, os números para lembrar e as perguntas prováveis de cada slide. O texto de "O que dizer" é o mesmo que está nas notas do apresentador no arquivo `.pptx`.
3. **Guia de estudo:** os conceitos que sustentam o trabalho, com exemplos usando os nossos próprios números.
4. **Perguntas difíceis:** respostas curtas e longas para as perguntas mais prováveis da banca.
5. **Consulta rápida:** a cola de números, o glossário e onde encontrar cada coisa no projeto.

Arquivos. `apresentacao/Apresentacao_TAES.pptx` (editável, com notas do apresentador) e `apresentacao/Apresentacao_TAES.pdf` (para projetar; serve de plano B). Os nomes dos apresentadores estão como "Apresentador 1" a "Apresentador 4". Para trocar, edite a lista `NOMES` no início de `apresentacao/fonte/build_slides.js` e gere de novo. Também dá para editar direto no PowerPoint, nos slides 1, 2, 7, 13, 22 e 29.

Parte 1 · Plano da apresentação

A mensagem em uma frase

> O verificador adversarial corta falsos alarmes, mas corta issues reais junto e não separa melhor o real do falso: ele **muda o ponto de operação**, não a qualidade da detecção.

Toda a apresentação serve para sustentar essa frase com números. Se alguém da plateia sair lembrando só dela, a apresentação funcionou.

Três ideias que a plateia deve levar

1. **O artigo mede só recall relativo.** O que ele chama de "acurácia" é "issues detectadas ÷ 56". Falso positivo nunca foi medido, e a auditoria achou inconsistências nas tabelas e no texto.
2. **O pipeline reduz falsos alarmes, mas perde detecção.** Os falsos alarmes conhecidos caem de 15,3% para 2,4%, e o recall cai de 19,6% para 7,1%. A H1 não foi confirmada, nem na execução principal nem nas 3 repetições.
3. **O verificador não discrimina os achados do detector.** Ele separa bem os achados de scanner (39% contra 5%), mas não os achados do detector (36% contra 15%, p = 0,36). E os números absolutos dependem muito do modelo usado.

Divisão do tempo

Bloco	Quem	Slides	Tempo	Mensagem do bloco
1. Contexto e auditoria	Apresentador 1	1 a 6	~8 min	O artigo mede só recall relativo e tem inconsistências; daí a nossa pergunta.
2. Proposta e método	Apresentador 2	7 a 12	~9 min	Três agentes, seis configurações, gabarito dos próprios autores, regra estrita, custo zero.
3. Resultados	Apresentador 3	13 a 21	~11 min	O verificador corta os dois lados; H1 não confirmada; o ganho é de ruído.
4. Análises e conclusão	Apresentador 4	22 a 29	~10 min	O modelo pesa nos números absolutos; H5 degenerada; limites; conclusão.
Perguntas	todos	apêndices 30 a 33	o restante	

Total previsto: **38 minutos**, o que deixa 2 minutos de folga para chegar a 40. Se a banca reservar tempo separado para perguntas, use a folga para falar mais devagar nos slides 15, 16 e 21, que são os mais importantes.

Passagens entre blocos

Combinem as frases de passagem e ensaiem as trocas. Uma troca bem feita leva 5 segundos; uma troca confusa leva 30.

- **Bloco 1 → 2** (fim do slide 6): "Passo para [Apresentador 2], que explica como montamos o experimento."
- **Bloco 2 → 3** (fim do slide 12): "Passo para [Apresentador 3], com os resultados."
- **Bloco 3 → 4** (fim do slide 21): "O ganho real é de ruído. Passo para [Apresentador 4]."

- Cada divisor (slides 7, 13 e 22) é aberto por quem assume: "Obrigado. Neste bloco eu mostro..." e a pessoa já passa para o slide seguinte.

Se o tempo apertar

Corte nesta ordem, sem pular slides (pular slides confunde a plateia):

1. Slide 14 (frase-resumo): diga só a frase, 15 segundos.
2. Slide 17 (H4): diga só "o verificador acusa 4 contra 1 do corte de confiança; evidência fraca".
3. Slide 12 (modelos): diga só os dois modelos e "custo zero, 17,6 horas de GPU gratuita".
4. Slide 27 (limites): cite só o gabarito e a amostra pequena.

Os slides que **não** devem ser encurtados são o 15 (resultado principal), o 16 (o verificador discrimina pouco), o 21 (balanço) e o 28 (conclusão).

Quem responde o quê nas perguntas

Tema da pergunta	Responde primeiro	Apoio
Artigo, DevGPT, auditoria, "isso invalida o artigo?"	Apresentador 1	4
Agentes, prompts, configurações, gabarito, regra de correspondência, métricas, estatística	Apresentador 2	3
Resultados, hipóteses H1 a H4, variância, balanço, custo	Apresentador 3	2
Modelos do artigo (E1), peso do modelo, H5, correção, limites, próximos passos	Apresentador 4	1

Regra simples: quem responde primeiro fala até 1 minuto; o apoio só complementa se faltar algo importante. Se ninguém souber, digam "não medimos isso" ou "não temos dado para afirmar". É melhor que inventar.

Checklist de ensaio

- ☐ Trocar "Apresentador 1–4" pelos nomes reais (veja "Arquivos", acima).
- ☐ Ensaiar com cronômetro, bloco por bloco; anotar o tempo real de cada um.
- ☐ Ensaiar as três passagens de bloco.
- ☐ Testar o arquivo no computador da sala: o `.pptx` usa a fonte Segoe UI (padrão do Windows). Em Mac ou no Google Slides a fonte é substituída; nesse caso, projete o PDF.
- ☐ Levar o PDF num pendrive e na nuvem (plano B).
- ☐ Cada pessoa sabe de cor os números do próprio bloco (Parte 5, "Cola de números").
- ☐ Cada pessoa leu a Parte 4 inteira.

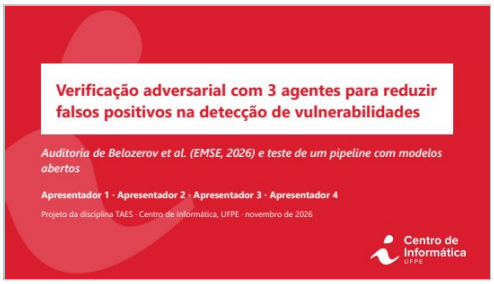
Dicas de fala

- **Não leia o slide.** O slide mostra o número; você explica o que ele quer dizer.
- **Diga os números também como contagem:** "4 de 56" é mais concreto que "7,1%".
- **Use sempre os mesmos termos:** "issue real" (uma das 56), "falso alarme conhecido" (um dos 85), "fora do gabarito" (não julgado). Não alterne com "verdadeiro positivo" ou "vulnerabilidade confirmada", para não confundir.
- **Não diga que o pipeline "é pior" ou "é melhor".** Diga que ele "opera em outro ponto": menos alarmes e menos detecção.
- **Seja honesto sobre os limites.** A banca valoriza quem sabe onde o próprio resultado para de valer.
- **Aponte para o gráfico** quando citar um número que está nele.

Parte 2 · Roteiro slide a slide

Cada slide traz: tempo previsto, quem apresenta, o que dizer (o mesmo texto das notas do `.pptx`), os números para lembrar e as perguntas prováveis. O texto de "O que dizer" é um roteiro, não para decorar palavra por palavra: entenda a ideia e fale do seu jeito.

Bloco 1 · Contexto e auditoria · Apresentador 1 · ~8 min



Slide 1 · Verificação adversarial com 3 agentes para reduzir falsos positivos na detecção de vulnerabilidades

Apresentador 1 · 0:45

O que dizer. Boa tarde. Somos Apresentador 1, Apresentador 2, Apresentador 3, Apresentador 4. Nosso trabalho parte de um artigo publicado em 2026 na Empirical Software Engineering, que avaliou LLMs na detecção e na correção de vulnerabilidades em código C, C++ e C#. Fizemos duas coisas: auditamos os números do artigo com os dados dos próprios autores e testamos uma proposta para reduzir falsos positivos, um pipeline de três agentes que roda só com modelos abertos e gratuitos.

Dica: apresente o grupo e diga logo que o trabalho tem duas partes, a auditoria do artigo e o experimento. Isso orienta a plateia.

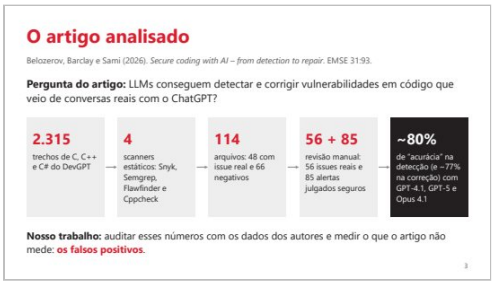


Slide 2 · Roteiro

Apresentador 1 · 0:45

O que dizer. A apresentação tem quatro blocos. Eu começo pelo contexto e pela auditoria do artigo. Apresentador 2 explica a proposta e o método. Apresentador 3 mostra os resultados. Apresentador 4 fecha com as análises complementares, os limites e a conclusão. A mensagem central, que vamos sustentar com números, é esta: o verificador corta falsos alarmes, mas corta issues reais junto, e não separa melhor o real do falso.

Dica: não leia os quatro cartões. Diga quem apresenta cada bloco e leia só a mensagem central, devagar.



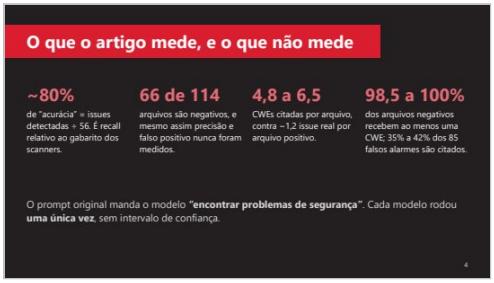
Slide 3 · O artigo analisado

Apresentador 1 · 1:45

O que dizer. O artigo pegou 2.315 trechos de C, C++ e C# do DevGPT, um conjunto de conversas reais de desenvolvedores com o ChatGPT, e passou esses trechos por quatro scanners estáticos. Daí saíram 114 arquivos. Na revisão manual, os autores confirmaram 56 vulnerabilidades reais em 48 arquivos e julgaram seguros 85 alertas dos scanners. Depois pediram a GPT-4.1, GPT-5 e Claude Opus 4.1, com um único prompt, que achassem e corrigissem os problemas. O resultado divulgado foi cerca de 80% de detecção e 77% de correção. Nosso trabalho foi auditar esses números e medir o que o artigo não mede: os falsos positivos.

Números para lembrar: 2.315 trechos; 4 scanners (Snyk, Semgrep, Flawfinder, Cppcheck); 114 arquivos (48 positivos, 66 negativos); 56 issues reais em 48 arquivos; 85 alertas julgados seguros; ~80% de detecção e ~77% de correção.

Se perguntarem "o que é o DevGPT?": um conjunto público de conversas de desenvolvedores com o ChatGPT, compartilhadas por link. O artigo usa dois snapshots: v8 (outubro de 2023, 63 arquivos) e v10 (maio de 2024, 51 arquivos).



Slide 4 · O que o artigo mede, e o que não mede

Apresentador 1 · 1:45

O que dizer. O que o artigo chama de acurácia é, na verdade, recall relativo: quantas das 56 issues do gabarito o modelo encontrou. Precisão e falso positivo nunca foram medidos, embora 66 dos 114 arquivos sejam negativos, ou seja, arquivos sem nenhuma vulnerabilidade real. Os indícios de falso positivo estão nos próprios dados: os modelos citam de 4,8 a 6,5 CWEs por arquivo, contra cerca de 1,2 issue real por arquivo positivo, e quase todos os arquivos negativos recebem pelo menos uma CWE. O prompt original manda encontrar problemas de segurança, o que incentiva o modelo a sempre achar alguma coisa. E cada modelo rodou uma única vez, sem intervalo de confiança.

Números para lembrar: "acurácia" = detectadas ÷ 56; 66 de 114 arquivos negativos; 4,8 a 6,5 CWEs citadas por arquivo; ~1,2 issue real por arquivo positivo (56 ÷ 48 = 1,17); 98,5% a 100% dos negativos com ao menos uma CWE; 35% a 42% dos 85 falsos alarmes citados; cada modelo rodou 1 vez.

Se perguntarem "por que recall relativo?": porque o gabarito só tem o que os scanners acharam. Um modelo que acha uma vulnerabilidade real que nenhum scanner viu não ganha nada nesse "recall".



Slide 5 · Auditoria: o que não bate no artigo

Apresentador 1 · 1:45

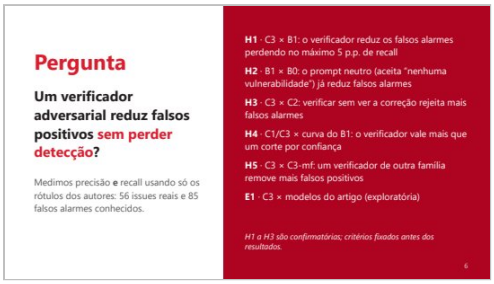
O que dizer. Recalculamos as tabelas do artigo a partir dos dados dos autores, sem usar LLM. Destaco seis pontos. Duas afirmações foram refutadas: o GPT-5 não corrigiu todas as 44 issues que detectou, porque detectou 44 e corrigiu 44, mas não as mesmas; e o Finding 15 não vale para GPT-4.1 nem para Opus 4.1. A melhora de 2024 para 2025 não é estatisticamente significativa. Há erros em tabelas, inconsistências no texto, e o gabarito mudou entre as duas versões no mesmo snapshot. Nada disso invalida o artigo, mas mostra que os números precisam ser lidos com cuidado.

Detalhe de cada item (todos recalculados a partir das tabelas do PDF e dos dados dos autores, sem LLM):

- **GPT-5 "corrigiu todas as 44 que detectou":** detectou 44 e corrigiu 44, mas os dois conjuntos não são os mesmos.
- **Finding 15:** diz que todos os modelos vão melhor em código do usuário do que em código gerado; a recontagem mostra que isso não vale para GPT-4.1 nem para Opus 4.1.
- **"Melhora" de 2024 para 2025:** McNemar exato nas 28 issues comuns dá p de 0,065 a 0,267; nenhuma é significativa.
- **Tabelas com erros:** Tabela 11 (texto diz 17 corrigidas e 15 não; a tabela tem 16 e 16); Tabelas 17 e 18 (3 de 6 células divergem); Tabela 14 (CWEs trocadas em `v10:3/7` e `v10:3/11`).
- **Inconsistências:** o texto fala em 3 scanners, o estudo usa 4; o resumo fala em reescanear o código corrigido, mas na p. 35 isso aparece como trabalho futuro; 49 das 56 issues vêm de um só scanner, embora o critério fale em "múltiplos scanners na mesma linha".

- **Gabarito mudou:** entre os rótulos de 2024 e 2025, no mesmo snapshot, 4 issues saíram e 2 mudaram de metadados.

Se perguntarem "isso invalida o artigo?": não. Mostra que a métrica tem o nome errado (recall relativo chamado de acurácia), que o falso positivo não foi medido e que algumas afirmações não se sustentam na recontagem.



Slide 6 · Pergunta

Apresentador 1 · 1:15

O que dizer. Daí a nossa pergunta: um verificador adversarial, ou seja, um agente cujo trabalho é tentar refutar cada achado, reduz falsos positivos sem perder detecção? Transformamos isso em hipóteses fixadas antes de rodar. A H1 é a principal: o pipeline completo reduz os falsos alarmes perdendo no máximo 5 pontos de recall. A H2 testa se só o prompt neutro já ajuda; a H3, se verificar antes de corrigir é melhor; a H4, se o verificador vale mais que um simples corte por confiança; a H5, se a família do verificador importa. A E1 compara com os modelos do artigo. Passo para Apresentador 2, que explica como montamos o experimento.

Para entender bem: a H1 é a hipótese principal. Ela exige duas coisas ao mesmo tempo: menos falsos alarmes **e** perda de recall de no máximo 5 p.p. Se uma das duas falha, a H1 não é confirmada.

Se perguntarem "por que só H1 a H3 são confirmatórias?": porque foram as comparações fixadas antes como principais. A H4 depende de uma curva e a H5 era opcional; a E1 compara modelos diferentes. Tudo o que não é confirmatório aparece rotulado como exploratório.

Bloco 2 · Proposta e método · Apresentador 2 · ~9 min



Slide 7 · Proposta e método

Apresentador 2 · 0:15

O que dizer. Obrigado. Neste bloco eu mostro a proposta e como medimos.

Dica: abra o bloco com uma frase e passe logo para o slide 8.



Slide 8 · Três papéis: detectar, corrigir, verificar

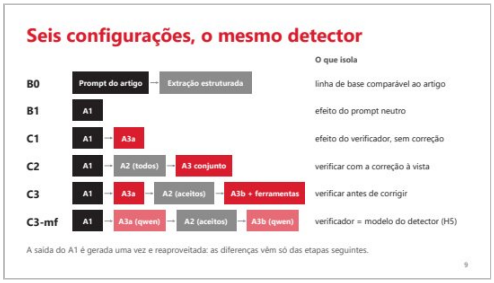
Apresentador 2 · 2:00

O que dizer. O pipeline tem três papéis. O A1, detector, recebe o código com linhas numeradas e devolve achados estruturados: a CWE, as linhas, de onde vem o dado do atacante, onde ele chega, qual o gatilho e uma confiança. Diferente do prompt do artigo, dizer "nenhuma vulnerabilidade" é uma resposta aceitável. O A2 é o corretor: gera um patch mínimo por achado. O A3 é o verificador adversarial, em duas versões. O A3a julga o achado tentando refutá-lo: o veredito padrão é "refutado", e ele só confirma se houver fonte não confiável, caminho até a operação perigosa e um gatilho concreto. O A3b julga a correção, com compilação e scanners. Regras importantes: o verificador vê um achado por vez, e nenhum agente vê o gabarito, a saída dos scanners ou a confiança do detector.

O A3a só confirma se as três condições forem verdadeiras: (1) existe uma fonte de dados não confiáveis plausível (stdin, arquivo, rede, argv, variável de ambiente, IPC, parâmetro de API pública); (2) existe um caminho da fonte até a operação perigosa que não é neutralizado por checagens do código; (3) existe um gatilho concreto. Antes de confirmar, ele precisa checar se há validação em outra linha, se os argumentos são constantes, se o caminho é inalcançável e se o item é só qualidade de código.

A3b: recebe o código original, o achado, o patch e o resultado das ferramentas: compilação só de sintaxe (`-fsyntax-only`) do original e do patch, e reescaneamento com flawfinder, cppcheck e semgrep. Veredito: `valid`, `partial` ou `invalid`.

Onde estão os prompts: `pipeline/prompts/a1_v2.md`, `a3_v1.md`, `a2_v1.md`, `a3b_v1.md`; o prompt original do artigo é `b0_paper.md` e o extrator do B0 é `b0_extract_v1.md`.



Slide 9 · Seis configurações, o mesmo detector

Apresentador 2 · 2:00

O que dizer. Com esses papéis montamos seis configurações. O B0 é o prompt original do artigo, rodando no nosso modelo, seguido de uma extração estruturada para podermos comparar. O B1 é só o detector. O C1 acrescenta o verificador. O C2 corrige tudo primeiro e depois verifica achado e correção juntos, que era a ordem da proposta original. O C3 verifica antes e só corrige o que foi confirmado. O C3-mf é a ablação da H5: o verificador passa a ser o mesmo modelo do detector. Um detalhe de desenho: a saída do A1 é gerada uma vez e reaproveitada, então qualquer diferença entre B1, C1, C2 e C3 vem só das etapas depois do detector.

Para entender bem: o B0 roda o prompt do artigo no nosso modelo. Como a resposta é texto livre, um segundo passo (o extrator, mesmo modelo) a converte em achados estruturados, para aplicarmos a mesma regra. No C3-mf, os achados do A1 vêm do cache da execução principal, então são idênticos aos do C3 (conferido nos 114 arquivos); só o verificador muda.

Se perguntarem "por que C2 era a ordem original?": a proposta inicial era detectar, corrigir e depois verificar. O C3 inverte: verifica antes e só corrige o que foi confirmado, o que evita patches desnecessários.



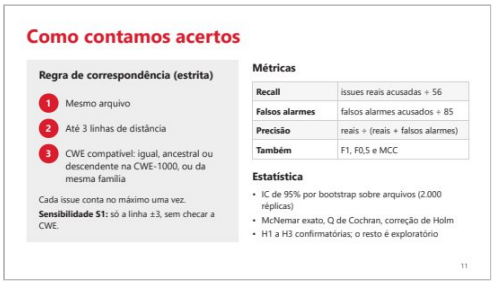
Slide 10 · Gabarito: só os rótulos dos autores

Apresentador 2 · 1:30

O que dizer. Não tínhamos um especialista para anotar, então usamos só o julgamento que os autores já tinham feito. São 114 arquivos: 48 com issue real e 66 negativos. E são 141 locais de código rotulados: 56 issues reais e 85 falsos alarmes conhecidos, que são alertas de scanner que os autores examinaram e julgaram seguros. Cada achado do pipeline cai em uma de três classes: casa com uma issue real, casa com um falso alarme conhecido, ou fica fora do gabarito. Essa terceira classe não é julgada: pode ser uma alucinação ou uma vulnerabilidade que o scanner não viu. Por isso a precisão é calculada só no conjunto rotulado.

Números para lembrar: 114 arquivos = 63 do v8 + 51 do v10; 48 positivos e 66 negativos; 141 locais rotulados = 56 reais + 85 falsos alarmes conhecidos. O arquivo `v8:File_02` fica de fora, como no artigo. Rótulos de 2025.

Se perguntarem "e os achados fora do gabarito?": podem ser alucinações ou vulnerabilidades reais que os scanners não viram. Sem anotador, não dá para saber. Por isso eles só aparecem nas descritivas: no B0 são 328 de 374 achados, no B1 202 de 229, no C3 16.



Slide 11 · Como contamos acertos

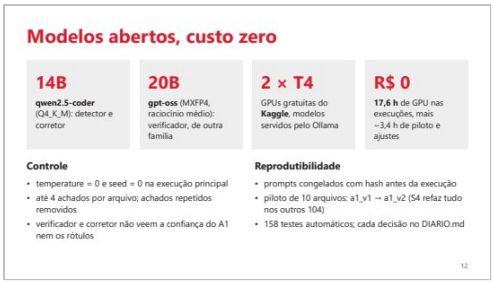
Apresentador 2 · 1:45

O que dizer. Para dizer que um achado casa com o gabarito, usamos uma regra estrita: mesmo arquivo, até três linhas de distância e CWE compatível, ou seja, igual, ancestral ou descendente na hierarquia oficial da MITRE, ou da mesma família. Cada issue conta uma vez. Como a regra pesa muito, rodamos análises de sensibilidade; a principal é a S1, que ignora a CWE. As métricas são o recall sobre as 56, a taxa de falsos alarmes sobre as 85 e a precisão no conjunto rotulado, além de F1, F0,5 e MCC. Os intervalos de confiança vêm de bootstrap sobre arquivos, e as comparações pareadas usam McNemar e Cochran com correção de Holm. Só H1 a H3 são confirmatórias.

CWE compatível quer dizer: a mesma CWE; ou uma ancestral ou descendente na hierarquia *CWE Research View 1000* da MITRE; ou da mesma família, segundo a tabela `matching_families.yaml`. Exemplo: CWE-120 (cópia de buffer sem checar tamanho) é compatível com CWE-119 (sua ancestral).

Cada issue conta uma vez: achados repetidos sobre algo já acusado ficam fora do numerador e do denominador.

Se perguntarem "por que bootstrap sobre arquivos?": porque os achados de um mesmo arquivo não são independentes. Reamostrar arquivos inteiros mantém essa dependência e dá intervalos mais honestos.



Slide 12 · Modelos abertos, custo zero

Apresentador 2 · 1:30

O que dizer. Tudo rodou com custo zero. Detector e corretor: qwen2.5-coder, de 14 bilhões de parâmetros, quantizado. Verificador: gpt-oss, de 20 bilhões, de outra família, com raciocínio médio. Usamos as GPUs gratuitas do Kaggle, duas T4, com o Ollama servindo os modelos: foram 17,6 horas de GPU nas execuções. Para controlar a variação, a execução principal usa temperatura zero e seed fixa; os prompts foram congelados com hash antes de rodar, e cada decisão ficou registrada num diário. Um ponto de transparência: ajustamos o prompt do detector num piloto de 10 arquivos, e a sensibilidade S4 refaz a análise sem esses arquivos. Passo para Apresentador 3, com os resultados.

Números para lembrar: 9,7 h de GPU na execução principal; 17,6 h no total das execuções (mais ~3,4 h de piloto e ajustes); R$ 0. Duas GPUs T4 do Kaggle por sessão, dois servidores Ollama.

Se perguntarem "por que esses modelos?": são abertos, cabem numa T4 de 16 GB depois de quantizados e são de famílias diferentes (qwen e gpt-oss), o que evita que o verificador repita os mesmos erros do detector.

Bloco 3 · Resultados · Apresentador 3 · ~11 min



Slide 13 · Resultados

Apresentador 3 · 0:15

O que dizer. Obrigado. Agora os resultados.

Dica: abra o bloco com uma frase e passe logo para o slide 14.

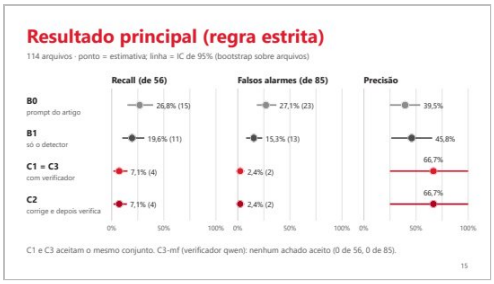


Slide 14 · Resultado em uma frase

Apresentador 3 · 0:45

O que dizer. Começo pelo resultado em uma frase. Dos falsos alarmes conhecidos que o detector acusava, o verificador removeu 85%. Mas também removeu 64% das issues reais. A H1 admitia perder no máximo 5 pontos de recall; perdemos 12,5.

Números para lembrar: 11 de 13 falsos alarmes removidos (85%); 7 de 11 issues reais perdidas (64%); recall caiu 12,5 p.p.; o limite da H1 era 5 p.p.



Slide 15 · Resultado principal (regra estrita)

Apresentador 3 · 1:45

O que dizer. Aqui estão as três métricas principais, com intervalos de 95%. O B0, com o prompt do artigo, acusa 27% das issues reais e 27% dos falsos alarmes. O B1, só o detector com prompt neutro, já cai para 15% de falsos alarmes, com 20% de recall. Com o verificador, em C1 e C3, os falsos alarmes vão para 2,4%, mas o recall vai para 7%: são 4 issues reais em 56. A precisão sobe para 67%, mas o intervalo vai de 25% a 100%, porque se apoia em 6 locais. O C2 fica igual ao C3. E o C3-mf, que veremos no bloco 4, não aceita nenhum achado.

Números para lembrar:

Config	Recall	Falsos alarmes	Precisão	F1	MCC
B0	26,8% (15/56)	27,1% (23/85)	39,5%	31,9%	−0,00
B1	19,6% (11/56)	15,3% (13/85)	45,8%	27,5%	0,06
C1 = C3	7,1% (4/56)	2,4% (2/85)	66,7%	12,9%	0,12
C2	7,1% (4/56)	2,4% (2/85)	66,7%	12,9%	0,12

Como ler o gráfico: cada ponto é a estimativa e a linha é o intervalo de 95%. Quando duas linhas se sobrepõem muito, a diferença pode ser acaso. Repare na linha do C3 em precisão: vai de 25% a 100%.



Slide 16 · Remove muito, discrimina pouco

Apresentador 3 · 1:45

O que dizer. Este gráfico mostra o que o verificador fez com os locais rotulados que o detector acusava. Eram 11 issues reais, e ele manteve 4. Eram 13 falsos alarmes, e ele manteve 2. Proporcionalmente, manteve mais reais que falsos, 36% contra 15%, mas com números tão pequenos o teste exato de Fisher dá p = 0,36: não dá para distinguir isso de um filtro que rejeita quase tudo. O contraste interessante: na calibração, sobre os achados que os scanners geram, o mesmo verificador discrimina bem, 39% contra 5%. Ou seja, a capacidade de separar depende do tipo de achado que ele recebe.

A tabela por trás do teste de Fisher:

	Mantidos pelo verificador	Removidos
Issues reais que o B1 acusava	4	7
Falsos alarmes que o B1 acusava	2	11

Fisher exato: p = 0,36. Ou seja, com 24 locais, uma diferença de 36% contra 15% é compatível com acaso.

Calibração: antes da execução, rodamos o A3a sobre os 141 achados reconstruídos a partir das saídas dos scanners (139 julgados). Ele confirmou 22 de 56 reais (39%) e 4 de 85 falsos alarmes (5%), $p = 3 \times 10^{-7}$. Essa sensibilidade de 39% já indicava que o limite de 5 p.p. da H1 seria difícil de cumprir.

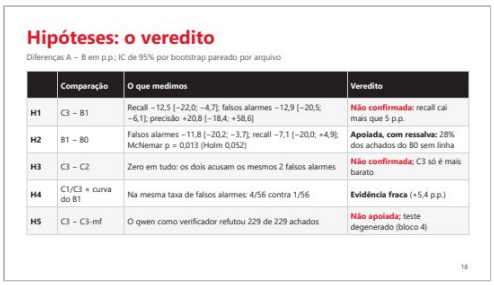


Slide 17 · Verificador × corte de confiança (H4)

Apresentador 3 · 1:15

O que dizer. A H4 pergunta se o verificador vale mais que um corte simples. A linha laranja é o B1 aceitando só achados com confiança acima de um limiar. No mesmo nível de falsos alarmes do C3, 2,4%, o melhor corte do B1 acusa 1 issue real; o verificador acusa 4. É evidência a favor do verificador, mas fraca: 4 contra 1 locais.

Para entender bem: a curva laranja é o que se consegue sem verificador, só exigindo mais confiança do detector. Cada ponto é um limiar. Se o verificador fosse só um "corte de confiança disfarçado", o ponto do C3 cairia em cima da curva. Ele fica um pouco acima (4 contra 1 issue real no mesmo nível de falsos alarmes), mas com tão poucos locais isso é evidência fraca.



Slide 18 · Hipóteses: o veredito

Apresentador 3 · 1:30

O que dizer. Resumindo as hipóteses. A H1 não foi confirmada: os falsos alarmes caem, mas o recall cai bem mais que 5 pontos e a precisão não melhora de forma significativa. A H2 foi apoiada com ressalva: só o prompt neutro já reduz os falsos alarmes em quase 12 pontos, sem perda significativa de recall; a ressalva é que 28% dos achados do B0 não têm linha. A H3 não foi confirmada: verificar antes ou depois de corrigir deu o mesmo resultado, e a vantagem do C3 é de custo. A H4 tem evidência fraca. A H5 fica para o bloco 4.

Para entender bem a H2: o B0 (prompt do artigo) acusa 23 dos 85 falsos alarmes; o B1 (prompt neutro), 13. O McNemar compara os locais acusados por um e não pelo outro: 12 só no B0 e 2 só no B1, p = 0,013 (0,052 depois da correção de Holm). A ressalva: 106 dos 374 achados do B0 (28%) não trazem linha e não podem casar pela regra estrita, o que pode baixar artificialmente os números do B0.

H3: C2 e C3 acusam exatamente os mesmos 2 falsos alarmes e as mesmas 4 issues reais (1 local só em cada um). A vantagem do C3 é de custo: 2 patches desnecessários contra 14, e 72 s contra 197 s de chamada por arquivo.



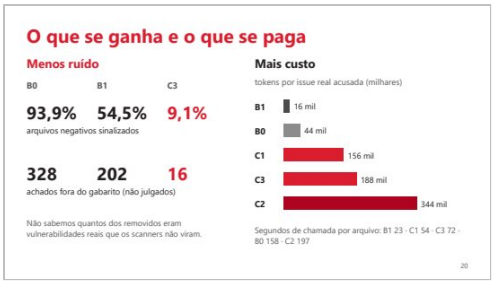
Slide 19 · Variância: a conclusão se repete

Apresentador 3 · 1:15

O que dizer. Como a execução principal é única, repetimos B1 e C3 três vezes com amostragem. Em nenhuma das três o critério da H1 foi cumprido. A queda de falsos alarmes é consistente, cerca de 12 pontos com intervalo abaixo de zero em todas, e a perda de recall também. Já a precisão do C3 oscila de 20% a 60%, porque se apoia em 5 a 7 locais. A execução principal fica dentro dessa variação.

Para entender bem: a execução principal usa temperatura 0. Nas repetições usamos a temperatura recomendada para cada modelo (0,7 no detector, 1,0 no verificador) e seeds 1, 2 e 3, para medir quanto o resultado varia. Arquivos negativos sinalizados: 50% a 58% no B1 e 12% a 14% no C3.

Se perguntarem "3 repetições bastam?": para estimar o desvio-padrão, não; é uma estimativa grosseira. Mas bastam para mostrar que a conclusão da H1 não depende de uma execução específica.



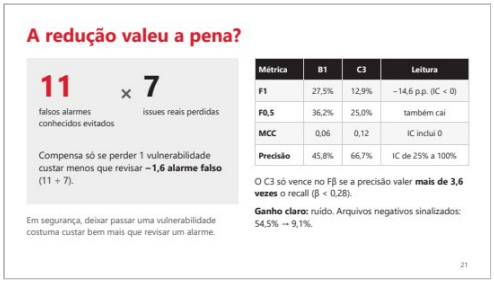
Slide 20 · O que se ganha e o que se paga

Apresentador 3 · 1:15

O que dizer. Há um ganho claro: ruído. O B0 sinaliza 94% dos arquivos negativos; o C3, 9%. E os achados fora do gabarito caem de 202 no B1 para 16 no C3. Mas não sabemos quantos desses eram reais, porque não houve anotador. E há custo: o C3 gasta cerca de 12 vezes mais tokens por issue real acusada do que o detector sozinho; o C2, 21 vezes.

Números para lembrar: arquivos negativos sinalizados: 93,9% (B0), 54,5% (B1), 9,1% (C3), 7,6% (C2). Achados fora do gabarito: 328 (B0), 202 (B1), 16 (C3). Tokens por issue real acusada: 16 mil (B1), 44 mil (B0), 156 mil (C1), 188 mil (C3), 344 mil (C2), ou seja, o C3 gasta ~12 vezes o B1 e o C2 ~21 vezes.

Cuidado ao falar: "arquivos negativos sinalizados" é um **limite superior** da taxa de falso positivo por arquivo, porque um achado num arquivo negativo pode ser uma vulnerabilidade real que os scanners não viram.



Slide 21 · A redução valeu a pena?

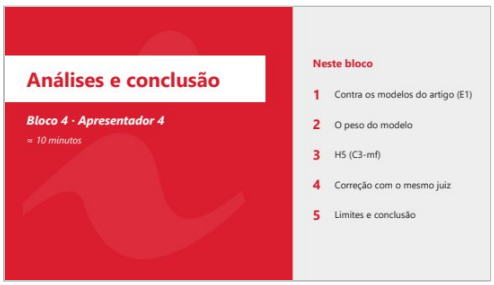
Apresentador 3 · 1:15

O que dizer. Então, a redução valeu a pena? Na conta absoluta, o C3 evita 11 falsos alarmes e perde 7 vulnerabilidades reais. Isso só compensa se perder uma vulnerabilidade custar menos do que revisar 1,6 alarme falso; em segurança, normalmente é o contrário. As métricas combinadas concordam: o F1 cai 14,6 pontos de forma significativa, e até o F0,5, que dá mais peso à precisão, cai. O C3 só venceria se a precisão valesse mais de 3,6 vezes o recall. O ganho real é de ruído. Passo para Apresentador 4.

De onde vem cada número: veja a seção 3.8 da Parte 3. Em resumo: 11 ÷ 7 ≈ 1,6; o F0,5 cai de 36,2% para 25,0%; o C3 só empata com o B1 no Fβ quando β ≈ 0,28, ou seja, quando a precisão vale ~3,6 vezes o recall.

Se perguntarem "e se os 186 achados fora do gabarito removidos fossem todos falsos?": aí a troca seria bem melhor (~197 alarmes evitados contra 7 reais perdidos). Mas não sabemos, porque não houve anotador. Essa é a principal coisa que um anotador humano resolveria.

Bloco 4 · Análises e conclusão · Apresentador 4 · ~10 min

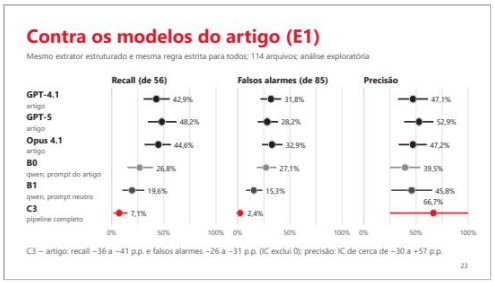


Slide 22 · Análises e conclusão

Apresentador 4 · 0:15

O que dizer. Obrigado. Neste último bloco mostro as análises complementares, os limites e a conclusão.

Dica: abra o bloco com uma frase e passe logo para o slide 23.



Slide 23 · Contra os modelos do artigo (E1)

Apresentador 4 · 1:30

O que dizer. Comparamos também com as respostas originais dos modelos do artigo, passando todas pelo mesmo extrator e pela mesma regra. GPT-4.1, GPT-5 e Opus acusam de 43% a 48% das issues reais e de 28% a 33% dos falsos alarmes. O C3 acusa 7% e 2%. Ou seja, o C3 tem muito menos falsos alarmes, mas só porque acusa muito menos de tudo; a precisão dele não se distingue da dos modelos comerciais. É outro ponto de operação, não um pipeline melhor.

Para entender bem: as respostas originais do artigo (texto livre) passam pelo mesmo extrator estruturado do B0 e pela mesma regra. Com a regra só-linha, os modelos do artigo chegam a 84% a 91% de recall, perto dos ~80% divulgados, o que mostra que a nossa reconstrução está coerente com o artigo. Mas, na mesma regra, eles acusam também 57% a 67% dos falsos alarmes conhecidos.

Ressalva: os modelos do artigo são outros e maiores, e uma extração do GPT-4.1 (`v8:File_04`) falhou e ficou sem achados, o que favorece levemente esse modelo na precisão.



Slide 24 · O peso do modelo

Apresentador 4 · 1:45

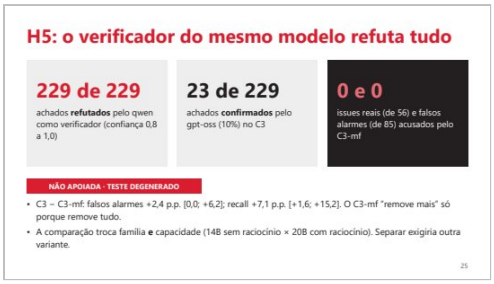
O que dizer. Usar modelos menores muda os resultados? Com o mesmo prompt, o qwen de 14B acusa de 16 a 21 pontos menos issues reais que os modelos do artigo, mas com taxa de falsos alarmes parecida. Olhando só a linha, os modelos maiores acham mais de tudo, reais e falsos, com a mesma precisão. Daí quatro conclusões: os números absolutos dependem do modelo; o modelo maior não resolve o falso positivo; o efeito do verificador depende de quem verifica; e em nenhuma troca o verificador separou melhor o real do falso. As comparações internas, como a H1, continuam válidas, porque usam o mesmo modelo em todas as configurações.

Os números da comparação B0 × modelos do artigo (mesmo prompt, mesmo extrator, mesma regra, mesmos 114 arquivos; diferença pareada, IC de 95%):

	contra GPT-4.1	contra GPT-5	contra Opus 4.1
Recall	−16,1 [−30,9; 0,0]	−21,4 [−35,4; −6,2]	−17,9 [−29,4; −4,3]
Falsos alarmes	−4,7 (IC inclui 0)	−1,2 (IC inclui 0)	−5,9 (IC inclui 0)
Precisão	−7,6 [−22,6; +6,3]	−13,5 [−27,1; +0,7]	−7,7 [−19,9; +5,0]

Só pela linha: B0 tem recall 67,9%, falsos alarmes 44,7% e precisão 50,0%; GPT-4.1, 83,9% / 60,0% / 48,0%; GPT-5, 91,1% / 67,1% / 47,2%; Opus 4.1, 83,9% / 56,5% / 49,5%. Os maiores acham mais de tudo, com a mesma precisão.

Por que a H1 continua válida: ela compara configurações que usam o mesmo detector e o mesmo verificador. O tamanho do modelo muda os números absolutos, não a comparação.



Slide 25 · H5: o verificador do mesmo modelo refuta tudo

Apresentador 4 · 1:15

O que dizer. A H5 pergunta se usar um verificador de outra família ajuda. No C3-mf, o verificador é o próprio qwen. Resultado: ele refutou os 229 achados, todos, com confiança alta e argumentos plausíveis. O gpt-oss tinha confirmado 23. Formalmente a H5 não é apoiada, mas o teste é degenerado: um verificador que rejeita tudo remove todos os falsos alarmes. E a comparação troca família e capacidade ao mesmo tempo, então não dá para concluir nada sobre erros correlacionados.

Detalhes: o qwen refutou os 229 achados com confiança de 0,8 a 1,0 e argumentos plausíveis; o gpt-oss confirmou 23 (10%) e refutou 87%. McNemar por local: issues reais 4 só no C3 e 0 só no C3-mf (p = 0,125); falsos alarmes 2 contra 0 (p = 0,5). O critério de análise da H5 foi registrado no `DIARIO.md` antes de rodar.

Como separar família de capacidade (não feito): usar um verificador de outra família sem raciocínio, ou rodar o qwen como verificador sobre os 141 locais da calibração, para ver se ele discrimina achados de scanner.



Slide 26 · Correção, com o mesmo juiz (A3b)

Apresentador 4 · 1:15

O que dizer. Na correção, usamos o mesmo juiz, o A3b, para todos. O B0 corrigiu 3 das 15 issues reais que acusou; C2 e C3, nenhuma das 4. Isso reflete mais quantas issues cada um acusa do que a qualidade do patch. Duas ressalvas: o juiz automático concorda pouco com os rótulos dos autores, kappa de 0,28; e o B0 gera um patch por arquivo, enquanto o pipeline gera um por achado. Não há evidência de que verificar antes de corrigir melhore a correção.

Critérios combinados de correção: veredito `valid` do A3b, a regra do scanner sumiu na linha, o patch compila e não aparece aviso novo. No B0, pela regra estrita: 15 issues acusadas, 5 com `valid`, 3 corrigidas pelos critérios combinados (20%; 5,4% das 56). Pela regra só-linha: 38 acusadas e 10 corrigidas (26,3%; 17,9% das 56).

κ (kappa) do juiz: calculado comparando o A3b com o rótulo "Was issue fixed" dos autores nos 56 patches do GPT-4.1 do artigo: 0,28 (só `valid` conta), 0,42 (se `partial` conta), 0,13 (só ferramentas). Concordância fraca, por isso a correção é análise secundária.



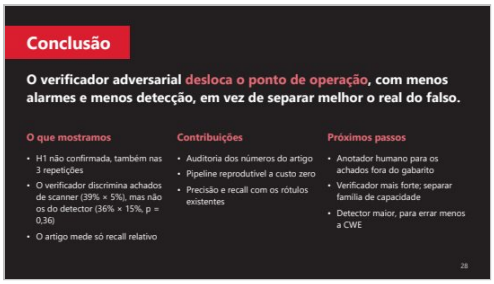
Slide 27 · Limites do estudo

Apresentador 4 · 1:30

O que dizer. Os limites mais importantes. O gabarito mistura riscos de padrão, como um strlen ou um printf que o scanner marca, com vulnerabilidades exploráveis, e o verificador procura explorabilidade; parte da perda pode ser rejeição correta, mas sem anotador não dá para afirmar. Os falsos alarmes conhecidos são só do tipo scanner. A amostra é pequena. A regra de correspondência concorda pouco com os rótulos de detecção dos autores, por isso fizemos sensibilidades. E os modelos são menores, com risco de contaminação, porque tudo é público.

Para entender bem o primeiro limite: das 11 issues reais que o B1 acusou, o A3a confirmou 4, refutou 6 e deu 1 "uncertain". Lendo o começo das 6 refutações, os argumentos parecem plausíveis (`strlen` não escreve fora do buffer; `strncpy` com terminação explícita; `printf` com formato constante). Isso poderia explicar parte da "perda de issues reais": o gabarito marca o risco do padrão, o verificador procura uma falha explorável. Sem anotador, não dá para afirmar quem está certo.

κ da regra de correspondência: de 0,08 a 0,27 contra os rótulos `detected` dos autores, que são lenientes (cerca de 80% "sim").



Slide 28 · Conclusão

Apresentador 4 · 2:00

O que dizer. Concluindo: neste experimento, o verificador adversarial desloca o ponto de operação, com menos alarmes e menos detecção, em vez de separar melhor o real do falso. A premissa de que mais um agente resolve o falso positivo sem perder detecção não se confirmou. O que fica de contribuição: a auditoria do artigo, um pipeline reprodutível a custo zero e uma avaliação que mede precisão e recall com os rótulos que já existiam. Como próximos passos: um anotador humano para os achados fora do gabarito, um verificador mais forte e uma variante que separe família de capacidade, e um detector maior.

Dica: este é o slide mais importante do bloco 4. Fale devagar a frase principal e só depois as três colunas. Termine olhando para a banca, não para a tela.



Slide 29 · Muito obrigado!

Apresentador 4 · 0:30

O que dizer. Obrigado. Ficamos à disposição para perguntas. Temos slides de apêndice com a sensibilidade, os custos, os testes pareados e a auditoria completa.

Dica: depois do "obrigado", deixe o slide parado. Se uma pergunta pedir detalhe, use os apêndices: A (sensibilidade), B (custo), C (testes pareados), D (auditoria completa).

Apêndice (para as perguntas)



Slide 30 · Apêndice A · Sensibilidade

Usar só se houver pergunta

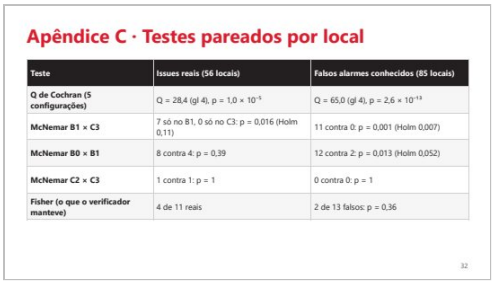
O que dizer. Para perguntas sobre a regra de correspondência. Cada célula é recall / taxa de falsos alarmes. A conclusão qualitativa não muda em nenhuma variante: o verificador corta os dois juntos. A S1 mostra quanto a regra estrita mede a CWE que o modelo escolhe: o B1 sobe de 19,6% para 55,4% de recall quando a CWE é ignorada, porque o detector usa CWE-78 em quase tudo.



Slide 31 · Apêndice B · Custo por configuração

Usar só se houver pergunta

O que dizer. Para perguntas sobre custo. Cada configuração é contada sozinha: o A1 e o A3a compartilhados entram em todas as que os usam. A execução principal inteira levou 9,7 horas de GPU; somando variância, E1, C3-mf e A3b no B0, foram 17,6 horas, sem pagamento.



Slide 32 · Apêndice C · Testes pareados por local

Usar só se houver pergunta

O que dizer. Para perguntas sobre os testes estatísticos. O Q de Cochran compara as cinco configurações ao mesmo tempo; o McNemar exato compara pares, contando os locais acusados por uma e não pela outra. A correção de Holm controla as comparações múltiplas.



Slide 33 · Apêndice D · Auditoria completa

Usar só se houver pergunta

O que dizer. Para perguntas sobre a auditoria. Todos os itens foram recalculados a partir das tabelas do PDF e dos dados dos autores, sem LLM. Detalhes em pipeline/results/auditoria_artigo.md.



Slide 34 · Encerramento (marca do CIn)

Usar só se houver pergunta

Parte 3 · Guia de estudo

3.1 O artigo em cinco minutos

Belozerov, Barclay e Sami (Empirical Software Engineering 31:93, 2026) perguntam se LLMs conseguem **detectar e corrigir** vulnerabilidades em código C, C++ e C# que apareceu em conversas reais com o ChatGPT (o conjunto DevGPT). O caminho foi:

1. Extrair 2.315 trechos de código do DevGPT.
2. Rodar quatro scanners estáticos (Snyk, Semgrep, Flawfinder, Cppcheck) e selecionar 114 arquivos.
3. Revisar à mão os alertas: 56 vulnerabilidades reais em 48 arquivos; os demais alertas (85 locais) foram julgados seguros, e 66 arquivos ficaram sem nenhuma vulnerabilidade real.
4. Pedir a GPT-4.1, GPT-5 e Claude Opus 4.1, com um único prompt ("You must find security issues..."), que achassem e corrigissem os problemas.
5. Contar quantas das 56 foram encontradas e corrigidas: "~80% de detecção, ~77% de correção".

O problema de método é que essa conta só olha para as 56. Um modelo que acusa tudo em todos os arquivos teria "acurácia" de 100%. O artigo não mede quantos alarmes falsos os modelos dão, embora tivesse os dados para isso (os 66 arquivos negativos e os 85 locais julgados seguros).

3.2 Conceitos de segurança

- **Vulnerabilidade:** uma falha que um atacante pode explorar. Exemplo: copiar uma entrada do usuário para um buffer sem checar o tamanho.
- **CWE (Common Weakness Enumeration):** catálogo da MITRE que dá um número a cada tipo de fraqueza. Exemplos que aparecem no projeto: CWE-20 (validação de entrada imprópria, a mais comum no gabarito), CWE-120 (cópia de buffer sem checar o tamanho), CWE-22 (*path traversal*), CWE-78 (injeção de comando do sistema operacional, que o nosso detector usa em quase tudo), CWE-561 (código morto) e CWE-1164 (código irrelevante). As duas últimas são de qualidade, não de segurança: por isso existe a sensibilidade S3, "só segurança".
- **Hierarquia CWE-1000 (*Research View*):** organiza as CWEs em árvore, das mais gerais às mais específicas. A regra de correspondência aceita uma CWE "parente" (ancestral ou descendente) da CWE do gabarito.
- **Scanner estático (SAST):** ferramenta que procura padrões perigosos no código sem executá-lo. Gera muitos alertas de padrão, por exemplo, todo `strcpy`, mesmo quando a origem do dado é segura.
- **Fonte, sink e gatilho:** a fonte é de onde vem o dado que o atacante controla (entrada, arquivo, rede); o sink é a operação perigosa (cópia de memória, comando do sistema); o gatilho é a entrada concreta que faz a falha acontecer. O verificador A3a só confirma um achado quando os três existem.
- **Risco de padrão × falha explorável:** um `printf(buf)` é um risco de padrão; só é uma falha explorável se `buf` vier do atacante. Parte da "perda de issues reais" pode vir dessa diferença entre o que o gabarito marca e o que o verificador procura.

3.3 LLMs e infraestrutura

- **Modelo aberto:** pesos publicados, que se pode rodar em máquina própria. Usamos `qwen2.5-coder:14b` (14 bilhões de parâmetros, especializado em código) e `gpt-oss:20b`.
- **Quantização:** guardar os pesos com menos bits para caber na GPU. Q4_K_M (qwen) e MXFP4 (gpt-oss) usam cerca de 4 bits por peso.
- **Raciocínio (*reasoning*):** o gpt-oss "pensa" antes de responder; fixamos o nível em "médio" em todas as configurações.

- **Ollama:** servidor que carrega o modelo e responde a chamadas. No Kaggle rodamos dois: um para o qwen e outro para o gpt-oss.
- **Kaggle:** plataforma com GPUs gratuitas (duas T4 de 16 GB por sessão), com cota semanal de cerca de 30 horas.
- **Temperatura e seed:** a temperatura controla o quanto a resposta varia. Com 0, o modelo escolhe sempre a palavra mais provável; a seed fixa o sorteio. Mesmo assim, respostas longas não são 100% reprodutíveis, por isso fizemos o estudo de variância.
- **Saída estruturada:** cada agente responde num JSON com formato fixo (um *schema*), validado por código. Se a resposta não segue o formato, a chamada é repetida até 2 vezes.
- **Prompt congelado:** o texto de cada prompt foi fixado, com hash registrado no `DIARIO.md`, antes da execução principal. Isso impede ajustar o prompt depois de ver o resultado.
- **Cache:** cada resposta fica guardada por uma chave que inclui modelo, prompt, parâmetros e entrada. Isso permitiu reaproveitar exatamente os mesmos achados do A1 em todas as configurações (e no C3-mf).
- **Contaminação:** como o DevGPT, o artigo e os dados são públicos, os modelos podem ter visto os rótulos no treino. Não testamos isso.

3.4 Desenho experimental

- **Configurações como experimento controlado:** todas partem do mesmo detector, com a mesma saída (gerada uma vez e reaproveitada). Assim, a diferença entre B1 e C3 só pode vir das etapas depois do detector.
- **Comparação pareada:** cada arquivo e cada local é avaliado em todas as configurações. Por isso usamos testes pareados (McNemar, bootstrap pareado), que são mais sensíveis que comparar médias soltas.
- **Um achado por chamada no verificador:** para que o julgamento de um achado não influencie o de outro.
- **O verificador não vê a confiança do detector:** senão ele poderia só copiar a confiança, e a H4 (verificador × corte de confiança) perderia o sentido.
- **Sem vazamento de gabarito:** nenhum agente que detecta ou verifica vê a saída dos scanners. Como o gabarito veio dos scanners, isso inflaria o recall.
- **Pré-registro:** hipóteses, critérios e prompts foram fixados antes dos resultados. Análises novas (como o peso do modelo) são rotuladas como exploratórias.
- **Piloto como conjunto de ajuste:** o prompt do detector foi ajustado olhando 10 arquivos (a versão 1 prejudicava o recall). Para não "ajustar no teste", a sensibilidade S4 repete a análise só nos outros 104.
- **Ablação:** remover ou trocar uma peça para ver o efeito dela. O C3-mf troca o verificador pelo próprio detector.

3.5 Métricas, com os nossos números

Considere o conjunto rotulado (141 locais) e o B1:

	Acusado pelo B1	Não acusado
Issue real (56)	VP = 11	FN = 45
Falso alarme conhecido (85)	FP = 13	VN = 72

- **Recall** = VP ÷ (VP + FN) = 11 ÷ 56 = **19,6%**. Das issues reais, quantas o sistema acusa.
- **Taxa de falsos alarmes conhecidos** = FP ÷ 85 = 13 ÷ 85 = **15,3%**. Dos locais seguros, quantos o sistema acusa por engano.
- **Precisão** = VP ÷ (VP + FP) = 11 ÷ 24 = **45,8%**. Dos acusados rotulados, quantos são reais.
- **F1** = média harmônica de precisão e recall = 2 × 0,458 × 0,196 ÷ (0,458 + 0,196) = **27,5%**.

- **Fβ:** generaliza o F1. Com β < 1 a precisão pesa mais; com β > 1 o recall pesa mais. O **F0,5** pesa a precisão mais que o recall. Fórmula: $F\beta = (1 + \beta^2)\,VP \div [(1 + \beta^2)\,VP + \beta^2\,FN + FP]$. Para o B1, F0,5 = 13,75 ÷ 38 = **36,2%**.
- **MCC (coeficiente de correlação de Matthews):** vai de −1 a +1; 0 é o mesmo que acusar ao acaso. Usa as quatro células: $MCC = (VP \cdot VN - FP \cdot FN) \div \sqrt{(VP+FP)(VP+FN)(VN+FP)(VN+FN)}$. No B1: (11·72 − 13·45) ÷ √(24·56·85·117) = 207 ÷ 3.656 ≈ **0,06**. No C3 (VP 4, FN 52, FP 2, VN 83): 228 ÷ 1.964 ≈ **0,12**. Os dois estão perto de zero: nenhuma configuração separa muito bem real de falso.

Métricas por arquivo: um arquivo está "sinalizado" se tiver pelo menos um achado aceito. A fração de **arquivos negativos sinalizados** (93,9% no B0, 54,5% no B1, 9,1% no C3) é um limite superior da taxa de falso positivo por arquivo.

Precisão geral com limites: se todos os achados fora do gabarito fossem falsos, a precisão geral seria o limite inferior; se todos fossem reais, o superior. No B1, de 4,9% a 94,2%. Um intervalo tão largo mostra o quanto falta um anotador.

3.6 Estatística

- **Intervalo de confiança (IC) de 95% por bootstrap:** reamostramos os 114 arquivos com reposição 2.000 vezes, recalculamos a métrica em cada amostra e pegamos os percentis 2,5% e 97,5%. Reamostrar **arquivos** (e não achados soltos) respeita o fato de que achados do mesmo arquivo não são independentes.
- **Bootstrap pareado da diferença:** em cada réplica, a mesma amostra de arquivos é usada nas duas configurações, e calculamos a diferença. Se o IC da diferença não inclui 0, a diferença é estatisticamente significativa a 5%.
- **McNemar exato:** compara duas configurações nos mesmos locais olhando só os **discordantes** (acusados por uma e não pela outra). Exemplo: issues reais acusadas só pelo B1 = 7, só pelo C3 = 0. Se não houvesse diferença, cada discordante teria 50% de chance de cair de cada lado; a chance de 7 contra 0 é $2 \times 0{,}5^7 \approx 0{,}016$.
- **Q de Cochran:** a versão do McNemar para mais de duas configurações ao mesmo tempo (aqui, cinco).
- **Correção de Holm:** quando se fazem vários testes, a chance de um "falso significativo" aumenta. Holm ajusta os p-valores para controlar isso. Por isso o 0,013 do H2 vira 0,052.
- **Fisher exato:** testa uma tabela 2 × 2 com contagens pequenas (como a do slide 16). p = 0,36 quer dizer que uma diferença como 4 de 11 contra 2 de 13 aparece com frequência por acaso.
- **κ de Cohen (kappa):** concordância entre dois avaliadores descontando a concordância por acaso. 0 é acaso e 1 é concordância total; valores abaixo de 0,4 costumam ser considerados fracos. O A3b teve 0,28 contra os autores.
- **Clopper-Pearson:** intervalo exato para uma proporção com poucas observações (usado na Figura 3 do relatório).
- **Confirmatória × exploratória:** um teste confirmatório foi planejado antes e responde a uma hipótese fixada; um exploratório é feito depois, para entender os dados, e serve para gerar hipóteses, não para prová-las.

3.7 Os resultados, explicados

- **Por que C1 = C3?** Nos dois, quem decide quais achados ficam é o A3a. O C3 só acrescenta a correção e o julgamento dela, que não muda o conjunto aceito. Então os dois têm os mesmos números de detecção por construção.
- **Por que C2 = C3 nos números?** Por coincidência: o A3 conjunto do C2 (que vê achado e correção) aceitou 4 issues reais e os mesmos 2 falsos alarmes. Há 1 local real só no C2 e 1 só no C3. A diferença está no custo.
- **Como a precisão sobe sem ser significativa?** Porque ela se apoia em 6 locais (4 reais e 2 falsos). Um local a mais ou a menos muda a precisão em mais de 10 p.p. O IC vai de 25% a 100%, e nas repetições a precisão do C3 foi de 20% a 60%.

- **"Reduz falsos alarmes" e "não discrimina" ao mesmo tempo?** Sim. Um filtro que rejeita quase tudo reduz os falsos alarmes, mas também as issues reais. Discriminar é rejeitar **mais** falsos do que reais, numa proporção que não seja acaso. O verificador manteve 36% dos reais e 15% dos falsos, mas com p = 0,36.
- **Por que o recall depende tanto da regra?** O detector usa CWE-78 em quase tudo. Pela regra estrita, um achado na linha certa com CWE errada não conta. Ignorando a CWE (S1), o recall do B1 sobe de 19,6% para 55,4%. A comparação entre configurações não muda, porque todas usam o mesmo detector.
- **E1:** contra os modelos do artigo, o C3 tem menos falsos alarmes (2,4% contra 28% a 33%), mas também muito menos recall (7% contra 43% a 48%), e a precisão não se distingue. É outro ponto de operação.
- **Peso do modelo:** com o mesmo prompt, o qwen acusa 16 a 21 p.p. menos issues reais que os modelos do artigo, com falsos alarmes parecidos. O modelo maior acha mais de tudo, com a mesma precisão. Logo, o falso positivo não é só problema de modelo pequeno, e o efeito do verificador depende de quem verifica.
- **H5:** o qwen como verificador refutou tudo. Um verificador que rejeita tudo "remove" todos os falsos alarmes, mas não diz nada sobre família de modelo. A comparação muda a família e a capacidade ao mesmo tempo.
- **Correção:** o B0 corrige mais em números absolutos porque acusa mais issues reais (15 contra 4). Não há evidência de que verificar antes de corrigir melhore a correção.

3.8 O balanço proporcional (slide 21)

Sobre os locais rotulados que o detector acusava, o C3 evitou 11 falsos alarmes e perdeu 7 issues reais.

- **Ponto de equilíbrio simples:** a troca compensa se o custo de perder uma vulnerabilidade for menor que o custo de revisar 11 ÷ 7 ≈ **1,6** alarmes falsos. Em segurança, deixar passar uma vulnerabilidade costuma custar muito mais que revisar um alarme.
- **Ponto de equilíbrio no Fβ:** o B1 tem VP 11, FN 45, FP 13; o C3, VP 4, FN 52, FP 2. Igualando os Fβ dos dois (com $b = \beta^2$): $11\,(1 + b) \div [11\,(1 + b) + 45\,b + 13] = 4\,(1 + b) \div [4\,(1 + b) + 52\,b + 2]$. Multiplicando cruzado: $11\,(6 + 56\,b) = 4\,(24 + 56\,b)$, ou seja, $66 + 616\,b = 96 + 224\,b$, logo $b = 30 \div 392 \approx 0{,}077$ e **$\beta \approx 0{,}28$**. O C3 só supera o B1 quando $\beta < 0{,}28$, isto é, quando a precisão vale mais de $1 \div 0{,}28 \approx$ **3,6 vezes** o recall.
- **O que as métricas combinadas dizem:** o F1 cai 14,6 p.p. (IC de −26,0 a −4,1, significativo); o F0,5, que já favorece a precisão, cai de 36,2% para 25,0%; o MCC vai de 0,06 para 0,12, com IC incluindo 0.
- **O ganho real:** ruído. Arquivos negativos sinalizados caem de 54,5% para 9,1%, e os achados fora do gabarito, de 202 para 16. Se esses 186 removidos fossem quase todos falsos, a troca seria muito melhor (~197 alarmes evitados contra 7 reais perdidos), mas sem anotador não sabemos.

Essa conta foi feita para a apresentação a partir dos números do relatório; ela não está no relatório final.

Parte 4 · Perguntas difíceis

Cada resposta tem uma versão curta (para dizer primeiro) e um complemento (se pedirem detalhe). Entre colchetes, quem responde primeiro.

1. Por que vocês não usaram os mesmos modelos do artigo? [Apresentador 2]
Curta: o projeto tinha orçamento zero; usamos só modelos abertos em GPUs gratuitas do Kaggle.
Complemento: para comparar com os modelos do artigo, usamos as respostas originais deles (E1), passando pelo mesmo extrator e pela mesma regra. As comparações internas (H1 a H3) usam o mesmo modelo em todas as configurações, então continuam válidas.

2. Sem anotador humano, como vocês sabem o que é falso positivo? [Apresentador 2]
Curta: só contamos como falso positivo os 85 locais que os próprios autores examinaram e julgaram seguros.
Complemento: o que não casa com o gabarito fica "fora do gabarito" e não é julgado. Por isso a precisão é calculada só no conjunto rotulado, e a fração de arquivos negativos sinalizados é apresentada como limite superior.

3. O recall baixo não é culpa do modelo pequeno? [Apresentador 4]
Curta: em parte, sim: com o mesmo prompt, o qwen acusa 16 a 21 p.p. menos issues reais que os modelos do artigo.
Complemento: mas isso não explica a H1, que compara configurações com o mesmo detector. O verificador perdeu 7 das 11 issues reais que o detector tinha achado; isso é efeito do verificador, não do detector.

4. A precisão do C3 foi 66,7%. Isso não mostra que o verificador funciona? [Apresentador 3]
Curta: são 4 acertos em 6 locais; o intervalo vai de 25% a 100%.
Complemento: a diferença para o B1 (+20,8 p.p.) tem IC de −18,4 a +58,6; nas repetições, a precisão do C3 oscilou de 20% a 60% e a diferença chegou a trocar de sinal. E o teste de Fisher sobre o que ele manteve dá p = 0,36.

5. E se o verificador estiver certo e o gabarito errado? [Apresentador 4]
Curta: é possível, e está nos limites.
Complemento: o gabarito marca riscos de padrão (`strlen`, `printf`) e o verificador procura falhas exploráveis. Das 6 issues reais que ele refutou, os argumentos parecem plausíveis. Mas sem um anotador não dá para afirmar que as rejeições estão certas.

6. A regra estrita, que exige a CWE certa, não é rígida demais? [Apresentador 2]
Curta: é rígida, e por isso fizemos sensibilidades.
Complemento: ignorando a CWE (S1), o recall do B1 sobe de 19,6% para 55,4%, porque o detector usa CWE-78 em quase tudo. Mas a conclusão qualitativa não muda: em S1, o verificador mantém 16% dos reais e 10% dos falsos. A regra também concorda pouco com os rótulos "detected" dos autores (κ de 0,08 a 0,27), que são lenientes.

7. Por que C1, C2 e C3 têm os mesmos números? [Apresentador 3]
Curta: C1 e C3 por construção (o mesmo A3a decide); C2 por coincidência.
Complemento: o C2 aceitou as mesmas 4 issues reais em contagem (1 local diferente de cada lado) e os mesmos 2 falsos alarmes. A diferença do C3 é de custo: 2 patches desnecessários contra 14 e 72 s contra 197 s por arquivo.

8. Temperatura zero garante que o resultado é reprodutível? [Apresentador 2]
Curta: não totalmente; por isso fizemos o estudo de variância.
Complemento: no piloto, só 1 de 10 respostas do B0 saiu idêntica entre execuções, mas os conjuntos de achados aceitos foram quase sempre iguais. Nas 3 repetições com amostragem, a conclusão da H1 se manteve em todas.

9. Por que só 3 repetições? [Apresentador 3]
Curta: limite de GPU gratuita; as 3 repetições já custaram 3,94 horas.

Complemento: 3 repetições não estimam bem o desvio-padrão, mas mostram que a conclusão não depende de uma execução específica. B0, C1 e C2 não foram repetidos.

10. Por que a H5 é "degenerada"? [Apresentador 4]
Curta: porque o verificador do mesmo modelo refutou todos os 229 achados.
Complemento: um verificador que rejeita tudo "remove" todos os falsos alarmes, mas não diz nada sobre erros correlacionados. Além disso, a troca mudou a família e a capacidade ao mesmo tempo (14B sem raciocínio contra 20B com raciocínio).

11. Por que o verificador discrimina na calibração e não nos achados do detector? [Apresentador 3]
Curta: não testamos a causa; é uma observação.
Complemento: uma explicação possível é que os achados de scanner trazem padrões que o prompt de refutação manda checar (argumento constante, checagem em outra linha), enquanto os achados do detector já chegam com uma narrativa de fonte até a operação perigosa, mais difícil de refutar ou confirmar. Isso seria um próximo passo.

12. Como vocês evitaram ajustar a análise ao resultado? [Apresentador 2]
Curta: hipóteses, critérios e prompts foram fixados antes, com registro no `DIARIO.md`.
Complemento: os prompts têm hash registrado; só H1 a H3 são confirmatórias; aplicamos Holm; tudo o que foi decidido depois está rotulado como exploratório. O único ajuste (prompt do detector no piloto) é declarado, e a S4 refaz a análise sem os arquivos do piloto.

13. Quanto isso custaria numa API paga? [Apresentador 3]
Curta: não calculamos o custo em dinheiro; medimos tokens.
Complemento: o C3 usa ~188 mil tokens por issue real acusada, contra ~16 mil do detector sozinho (cerca de 12 vezes); o C2, ~344 mil (cerca de 21 vezes). Com o preço de uma API, a conta é direta.

14. Os 85 falsos alarmes representam os falsos positivos de um LLM? [Apresentador 4]
Curta: não. São falsos alarmes do tipo que scanners geram.
Complemento: as alucinações livres do modelo (achados sem relação com o que o scanner viu) ficam fora do gabarito. Por isso mostramos também os arquivos negativos sinalizados e os achados fora do gabarito.

15. A auditoria invalida o artigo? [Apresentador 1]
Curta: não; ela mostra que a métrica está mal nomeada e que algumas afirmações não se sustentam.
Complemento: o artigo mede recall relativo, não acurácia; não mede falso positivo; tem inconsistências de tabela e de texto; e a "melhora" entre 2024 e 2025 não é significativa. A contribuição do artigo, o conjunto de dados rotulado, foi justamente o que tornou o nosso trabalho possível.

16. Por que usar o gpt-oss como verificador? [Apresentador 2]
Curta: é de outra família que o detector, cabe numa T4 e tem raciocínio.
Complemento: usar outra família era uma recomendação do plano, para reduzir erros em comum entre detector e verificador. A H5 tentou medir esse efeito, sem sucesso.

17. O que é "ponto de operação"? [Apresentador 3]
Curta: a combinação de recall e taxa de falsos alarmes em que um sistema funciona.
Complemento: quase todo detector pode trocar um pelo outro (mais rigoroso: menos alarmes e menos detecção). A curva do B1 no slide 17 mostra essa troca variando o corte de confiança. Um sistema "melhor" é o que fica acima da curva, não o que só anda sobre ela.

18. O que vocês fariam com mais recursos? [Apresentador 4]
Curta: um anotador humano para os achados fora do gabarito.
Complemento: também um verificador mais forte (para ver se perde menos issues reais), uma variante que separe família de capacidade, um detector maior (para errar menos a CWE) e repetições de B0, C1 e C2.

19. Qual é, afinal, a contribuição do trabalho? [Apresentador 4]
Curta: mostrar, com números, que o verificador adversarial muda o ponto de operação em vez de melhorar a

discriminação.
Complemento: e também a auditoria do artigo, um pipeline reprodutível a custo zero (prompts congelados, cache, 158 testes, diário de decisões) e um jeito de medir precisão e recall usando só os rótulos que já existiam.

20. Por que vocês chamam de "issue" e não de "vulnerabilidade"? [Apresentador 1]
Curta: porque é o termo do artigo, e o gabarito inclui alguns itens de qualidade de código.
Complemento: duas das 56 são de qualidade (CWE-561 e CWE-1164); a sensibilidade S3 refaz a análise só com as 54 de segurança, e nada muda.

Parte 5 · Consulta rápida

Cola de números

Tema	Números
Dados	114 arquivos (48 positivos, 66 negativos; 63 do v8, 51 do v10); 56 issues reais; 85 falsos alarmes conhecidos; 141 locais rotulados
Artigo	2.315 trechos; 4 scanners; ~80% de detecção, ~77% de correção; 4,8 a 6,5 CWEs citadas por arquivo; 98,5% a 100% dos negativos com CWE
B0	recall 26,8% (15/56); falsos alarmes 27,1% (23/85); precisão 39,5%
B1	recall 19,6% (11/56); falsos alarmes 15,3% (13/85); precisão 45,8%; F1 27,5%; F0,5 36,2%; MCC 0,06
C1 = C3	recall 7,1% (4/56); falsos alarmes 2,4% (2/85); precisão 66,7% [25; 100]; F1 12,9%; F0,5 25,0%; MCC 0,12
H1 (C3 − B1)	recall −12,5 [−22,0; −4,7]; falsos alarmes −12,9 [−20,5; −6,1]; precisão +20,8 [−18,4; +58,6]
Verificador	removeu 11/13 falsos (85%) e 7/11 reais (64%); manteve 4/11 e 2/13; Fisher p = 0,36
Calibração	confirmou 22/56 reais (39%) e 4/85 falsos (5%); $p = 3 \times 10^{-7}$
H2 (B1 − B0)	falsos alarmes −11,8 [−20,2; −3,7]; McNemar p = 0,013 (Holm 0,052)
H3	C3 = C2 nas métricas; 2 contra 14 patches desnecessários; 72 s contra 197 s por arquivo
H4	4/56 (C3) contra 1/56 (corte de confiança) com 2,4% de falsos alarmes
Variância	critério da H1 em 0 de 3; falsos alarmes −11,8 a −12,9; recall −7,1 a −12,5; precisão do C3 de 20% a 60%
Ruído	negativos sinalizados 93,9% / 54,5% / 9,1% (B0 / B1 / C3); fora do gabarito 328 / 202 / 16
Custo	tokens por issue real: 16 mil (B1), 188 mil (C3), 344 mil (C2); GPU 17,6 h + ~3,4 h; R$ 0
Balanço	11 evitados × 7 perdidos (1,6); β de equilíbrio ≈ 0,28 (3,6×)
E1	artigo: recall 43% a 48%, falsos alarmes 28% a 33%, precisão 47% a 53%; C3 − artigo: recall −36 a −41 p.p., falsos −26 a −31 p.p.
Peso do modelo	B0 × artigo: recall −16 a −21 p.p.; falsos alarmes −1 a −6 p.p. (IC inclui 0)
H5	qwen refutou 229/229; gpt-oss confirmou 23/229 (10%); C3-mf: 0/56 e 0/85
Correção	B0 3/15 (20%; 5,4% das 56); C2 e C3 0/4; κ do A3b = 0,28

Glossário

Termo	Significado
A1, A2, A3a, A3b	Detector, corretor, verificador do achado e verificador da correção
A3 conjunto	Verificador do C2, que julga achado e correção numa resposta só
Ablação	Experimento que remove ou troca uma peça para medir o efeito dela (C3-mf)
Achado	Uma acusação feita pelo sistema: arquivo, linhas, CWE
Bootstrap	Reamostragem com reposição para estimar a incerteza de uma métrica

Termo	Significado
Calibração	Teste do A3a sobre os achados reconstruídos dos scanners, antes da execução principal
Confirmatória	Análise planejada antes, que testa uma hipótese fixada (H1 a H3)
CWE	Catálogo de tipos de fraqueza de software, mantido pela MITRE
DevGPT	Conjunto público de conversas de desenvolvedores com o ChatGPT
E1	Comparação exploratória com as respostas dos modelos do artigo
Exploratória	Análise feita para entender os dados; não prova hipótese
Falso alarme conhecido	Local que os autores examinaram e julgaram seguro (um dos 85)
Fora do gabarito	Achado que não casa com nenhum local rotulado; não é julgado
Gabarito	Os rótulos dos autores: 56 issues reais e 85 falsos alarmes
Issue real	Vulnerabilidade confirmada pelos autores (uma das 56)
κ (kappa)	Concordância entre avaliadores descontando o acaso
McNemar	Teste pareado que olha só os casos discordantes
MCC	Correlação entre o que foi acusado e o que é real; 0 = acaso
Ponto de operação	O par (recall, taxa de falsos alarmes) em que o sistema funciona
p.p.	Pontos percentuais: diferença entre duas porcentagens
Precisão	Dos acusados rotulados, a fração que é real
Recall	Das issues reais, a fração acusada
Regra estrita	Mesmo arquivo, até 3 linhas e CWE compatível
Regra só-linha (S1)	Mesmo arquivo e até 3 linhas, sem checar a CWE
Verificador adversarial	Agente cujo papel é tentar refutar cada achado

Onde encontrar cada coisa no projeto

O quê	Onde
Relatório final	`relatorio/RELATORIO.pdf` (fonte: `relatorio/RELATORIO.md`)
Plano original, hipóteses e especificação dos agentes	`PLANO_EXECUCAO.md` (seções 1, 3 e 4)
Diário de decisões, hashes dos prompts, horas de GPU	`pipeline/DIARIO.md`
Prompts dos agentes	`pipeline/prompts/` (`a1_v2.md`, `a2_v1.md`, `a3_v1.md`, `a3b_v1.md`, `b0_paper.md`, `b0_extract_v1.md`)
Código dos agentes	`pipeline/src/agents/` (`detector.py`, `fixer.py`, `verifier.py`)
Orquestração das configurações	`pipeline/src/runner.py`
Modelos e parâmetros	`pipeline/config.yaml`

O quê	Onde
Métricas e testes	`pipeline/src/f7_analyze.py`; resultados em `pipeline/results/f7_resultados.md` e `metrics.csv`
Auditoria do artigo	`pipeline/results/auditoria_artigo.md`
Figuras	`pipeline/results/figs/`
Slides (gerador)	`apresentacao/fonte/build_slides.js`; fundos em `apresentacao/fonte/assets/`